

LangChain: 快速入门与底层原理

解锁大语言模型的应用开发潜力

本次分享大纲



1. 初识 LangChain: 它是什么, 为何重要?



2. 核心揭秘: 深入架构与工作原理

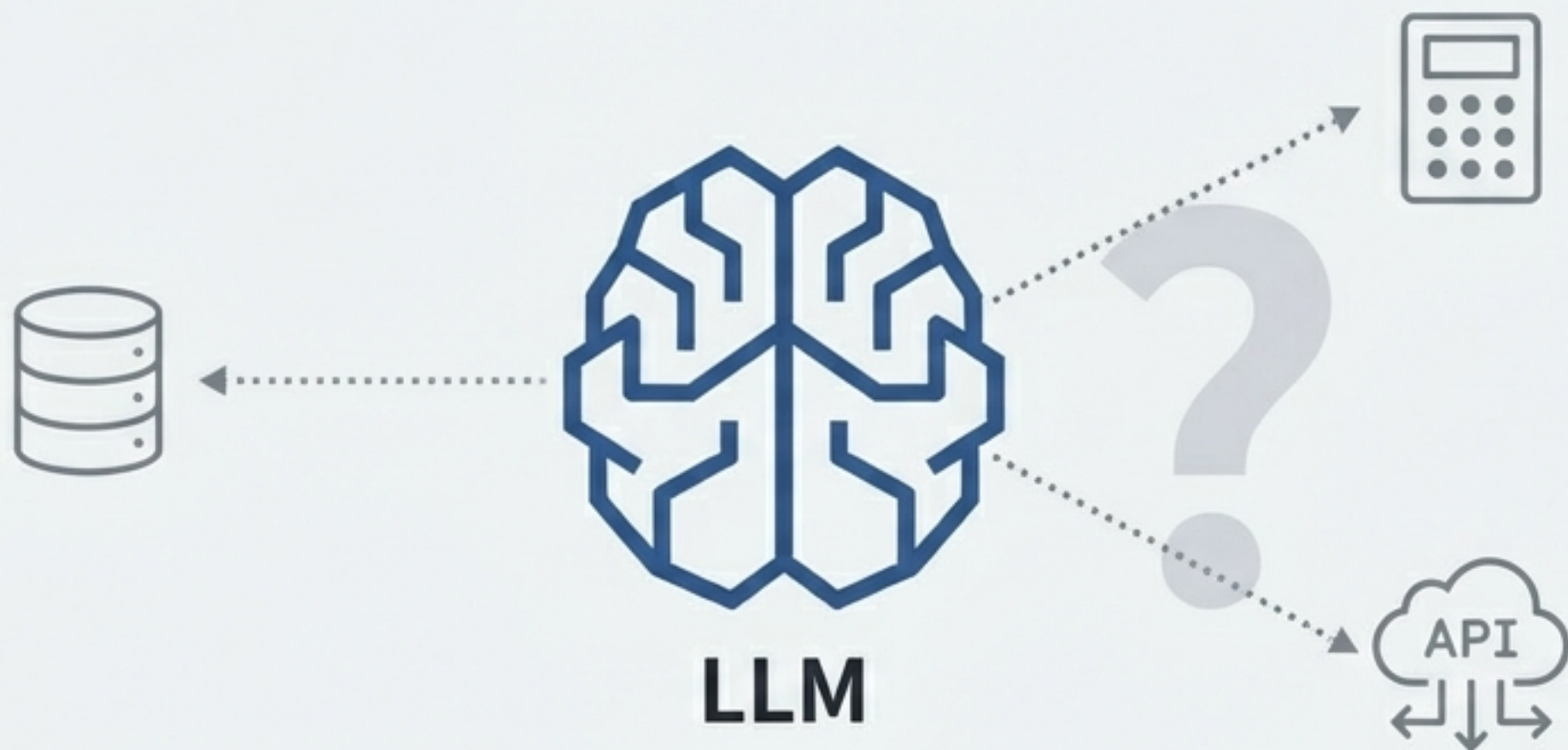


3. 动手实践: 构建你的第一个 LangChain 应用



4. 应用展望: 探索未来应用场景

挑战：从语言模型到真正的 AI 应用



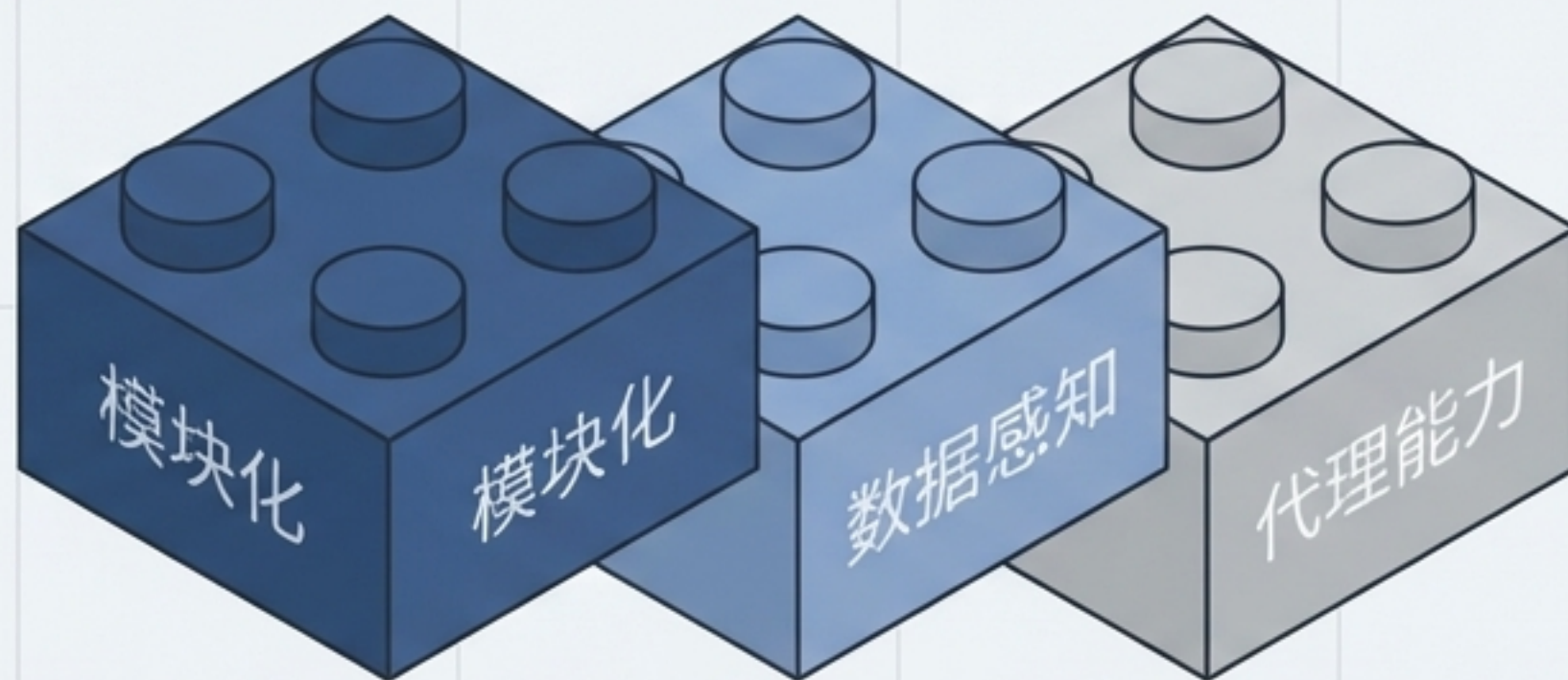
大语言模型（LLM）本身是强大的“大脑”，但它们无法主动与外部数据、工具或服务交互。

要构建能够解决现实世界问题的复杂应用，我们必须为模型提供与外界连接的桥梁。

LangChain：构建 AI 应用的开源编排框架

LangChain 是一个开源的 Python AI 应用开发框架，它提供了构建基于大模型的 AI 应用所需的模块和工具。通过 LangChain，开发者可以轻松地与大型语言模型（LLM）集成，完成文本生成、问答、翻译、对话等任务。

LangChain 降低了 AI 应用开发的门槛，让任何人都可以基于 LLM 构建属于自己的创意应用。



LangChain 的核心特性：为模型装上“手”和“脚”

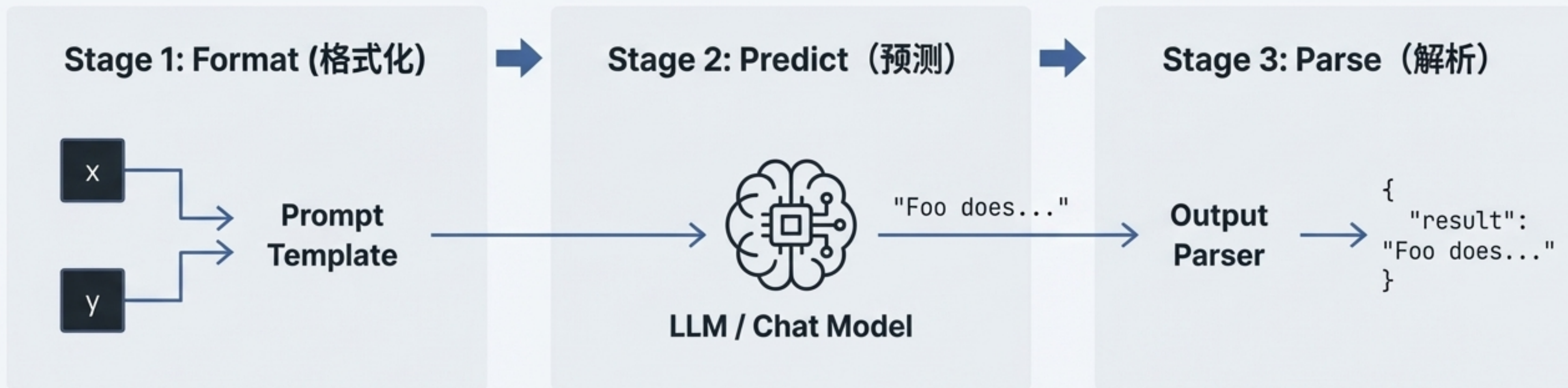


就是为模型装上了手和脚

架构一览：LangChain 的分层生态系统



一次请求的生命周期：从输入到结构化输出



使用提示词模板（Prompt Template）将用户的原始输入（如变量 `x`，`y`）格式化为对 LLM 友好的标准提示。

将格式化后的提示发送给 LLM 或聊天模型（Chat Model）进行处理。

使用输出解析器（Output Parser）将 LLM 返回的原始文本（如 `"Foo does..."`）转换为结构化的、可用的格式（如 JSON）。

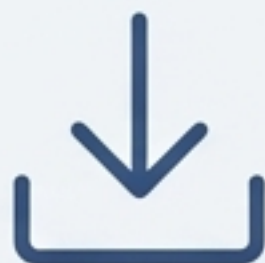
动手实践：构建你的第一个 LangChain 应用

四个步骤，从零到一

成功蓝图：构建 LangChain 应用的四个步骤

01

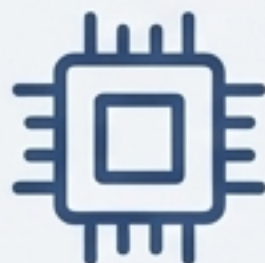
安装 (Install)



```
`pip install langchain  
langchain-community`
```

02

初始化模型
(Initialize Model)



配置 API 密钥并选择一个 LLM（例如：`ChatTongyi`）。

03

构建链
(Build a Chain)



使用 LangChain 表达式语言 (LCEL) 的 `|` 符号将提示模板和模型连接起来。

04

执行与转换
(Invoke & Parse)



调用链，传入输入，并使用解析器获得干净的输出。

实战演练（1/3）：定义提示与模型

```
# 1. 导入所需模块
from langchain_community.chat_models import ChatTongyi
from langchain_core.prompts import ChatPromptTemplate

# 2. 初始化模型（假设API密钥已在环境中配置）
llm = ChatTongyi()

# 3. 创建提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一位世界级的技术专家。"),
    ("user", "{input}")
])
```

System Role: 定义AI的身份和行为准则，为对话设定基调。

User Role: 用户的实际输入占位符，将在调用时被具体问题替换。

实战演练（2/3）：使用 LCEL 构建链并调用

```
# ... (代码从上一页继续)
```

```
# 4. 使用 LCEL 构建链
```

```
chain = prompt | llm
```

```
# 5. 调用链并传入输入
```

```
result = chain.invoke({"input": "帮我写一篇关于AI的技术文章"})
```

```
# 打印原始结果（通常是一个包含元数据的AIMessage对象）
```

```
print(result)
```

LCEL 的魅力所在：像管道一样连接组件

Invoke：执行链，将输入数据传递给第一个组件（Prompt Template）。

实战演练（3/3）：添加解析器以获得结构化输出

```
# ... (代码从上一页继续)
from langchain_core.output_parsers import StrOutputParser

# 6. 初始化输出解析器，将结果转换为纯字符串
output_parser = StrOutputParser()

# 7. 重新构建链，加入输出解析器
chain = prompt | llm | output_parser

# 8. 再次调用，这次将直接返回干净的字符串结果
result_str = chain.invoke({"input": "帮我写一篇关于AI的技术文章"})
print(result_str)
```

模块化：新的组件（输出解析器）被无缝地“插入”到链的末端，展示了 LangChain 的可组合性。

Clean Output：调用现在返回一个可以直接使用的字符串，而不是一个复杂的对象。

LangChain 的无限可能：典型应用场景

对话机器人



对话机器人

- 构建智能对话助手，提供实时对话服务、解答问题。
- 开发聊天机器人，用于社交、娱乐、语言学习等场景。

知识库问答



知识库问答

- 与知识图谱结合，进行开放域问题的问答服务。
- 用于企业内部知识管理，快速检索和回答员工问题。

智能写作



智能写作

- 协助用户撰写文章，提供写作思路、生成文章框架。
- 激发用户创意，生成创意性内容，如故事、诗歌等。

核心要点回顾

- ✓ **它是什么？**：LangChain 是一个**强大的开源框架**，用于编排和构建基于 LLM 的复杂应用程序。
- ✓ **为何强大？**：它的核心优势在于其**模块化组件**（Chains, RAG, Agents）和富有表现力的**LCEL声明式语法**。
- ✓ **如何开始？**：只需**几行代码**，你就可以利用 LangChain 的力量，构建并运行你的第一个 AI 应用。